UTEC
Universidad Tecnológica

Patrón DAO

El **Patrón DAO** (Data Access Object) es un patrón de diseño estructural que se utiliza para separar la lógica de acceso a datos de la lógica de negocio. Este patrón abstrae y encapsula todas las interacciones con una fuente de datos, como una base de datos, para que la lógica de negocio no dependa directamente de cómo se acceden o almacenan los datos. El DAO actúa como un intermediario entre la aplicación y la base de datos.

Ventajas del Patrón DAO:

1. **Separación de responsabilidades:** La lógica de acceso a datos se separa del resto de la aplicación, lo que facilita el mantenimiento y evolución del código.
2. **Facilidad para cambiar la fuente de datos:** Al usar un DAO, se puede cambiar la fuente de datos (por ejemplo, de MySQL a PostgreSQL) sin tener que modificar la lógica de negocio.
3. **Pruebas más sencillas:** Los DAOs se pueden probar de forma aislada, y es más fácil crear pruebas unitarias de la lógica de negocio al desacoplarla del acceso a los datos.

Ejemplo con una Entidad `Usuario`

Supongamos que tenemos una clase `Usuario` con los siguientes atributos:

```
public class Usuario {
    private int id;
    private String nombre;
    private String email;
    private String password;

    public Usuario(int id, String nombre, String email, String password)
{
        this.id = id;
        this.nombre = nombre;
        this.email = email;
        this.password = password;
    }

    // Getters y Setters
```

UTEC
Universidad Tecnológica

```
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password;
}
}
```

Interfaz DAO

El primer paso es definir una **interfaz DAO** que contenga los métodos para manipular los objetos `Usuario` en la base de datos.

```
import java.util.List;

public interface UsuarioDAO {
    void agregarUsuario(Usuario usuario);
    Usuario obtenerUsuarioPorId(int id);
    List<Usuario> obtenerTodosLosUsuarios();
    void actualizarUsuario(Usuario usuario);
    void eliminarUsuario(int id);
}
```

Implementación de `UsuarioDAO`

Ahora implementamos la interfaz `UsuarioDAO`. Para simular el almacenamiento de los usuarios, usaremos una lista en memoria.

```
import java.util.ArrayList;
import java.util.List;
```

UTEC
Universidad Tecnológica

```
public class UsuarioDAOImpl implements UsuarioDAO {
    private List<Usuario> usuarios = new ArrayList<>();

    @Override
    public void agregarUsuario(Usuario usuario) {
        usuarios.add(usuario);
        System.out.println("Usuario agregado: " + usuario);
    }

    @Override
    public Usuario obtenerUsuarioPorId(int id) {
        for (Usuario usuario : usuarios) {
            if (usuario.getId() == id) {
                return usuario;
            }
        }
        System.out.println("Usuario no encontrado con ID: " + id);
        return null;
    }

    @Override
    public List<Usuario> obtenerTodosLosUsuarios() {
        return usuarios;
    }

    @Override
    public void actualizarUsuario(Usuario usuario) {
        for (Usuario u : usuarios) {
            if (u.getId() == usuario.getId()) {
                u.setNombre(usuario.getNombre());
                u.setEmail(usuario.getEmail());
                System.out.println("Usuario actualizado: " + u);
                return;
            }
        }
        System.out.println("Usuario no encontrado para actualizar: " +
usuario);
    }

    @Override
    public void eliminarUsuario(int id) {
```

UTEC
Universidad Tecnológica

```
        usuarios.removeIf(u -> u.getId() == id);
        System.out.println("Usuario eliminado con ID: " + id);
    }
}
```

Uso del DAO

Aquí tienes un ejemplo de cómo utilizar el DAO para agregar, actualizar, obtener y eliminar usuarios. Simulamos un sistema que trabaja con usuarios sin la necesidad de usar una base de datos:

```
public class Main {
    public static void main(String[] args) {
        UsuarioDAO usuarioDAO = new UsuarioDAOImpl();

        // Crear usuarios
        Usuario usuario1 = new Usuario(1, "Juan Perez",
"juan@example.com");
        Usuario usuario2 = new Usuario(2, "Ana Gomez",
"ana@example.com");

        // Agregar usuarios
        usuarioDAO.agregarUsuario(usuario1);
        usuarioDAO.agregarUsuario(usuario2);

        // Obtener todos los usuarios
        System.out.println("Todos los usuarios:");
        for (Usuario usuario : usuarioDAO.obtenerTodosLosUsuarios()) {
            System.out.println(usuario);
        }

        // Actualizar un usuario
        Usuario usuarioActualizado = new Usuario(1, "Juan Perez",
"juan.perez@example.com");
        usuarioDAO.actualizarUsuario(usuarioActualizado);

        // Obtener un usuario por ID
        Usuario usuarioObtenido = usuarioDAO.obtenerUsuarioPorId(1);
        System.out.println("Usuario obtenido por ID: " +
```

UTEC
Universidad Tecnológica

```
usuarioObtenido);

        // Eliminar un usuario
        usuarioDAO.eliminarUsuario(2);

        // Mostrar usuarios restantes
        System.out.println("Usuarios restantes:");
        for (Usuario usuario : usuarioDAO.obtenerTodosLosUsuarios()) {
            System.out.println(usuario);
        }
    }
}
```

Explicación del código

1. **Entidad `Usuario`**: Es el objeto que queremos gestionar. Tiene propiedades como `id`, `nombre`, y `email`.
2. **Interfaz `UsuarioDAO`**: Define las operaciones necesarias para gestionar los usuarios, como agregar, obtener, actualizar y eliminar.
3. **Implementación `UsuarioDAOImpl`**: Implementa la lógica para gestionar los usuarios en una lista en memoria. En un caso real, esta lista sería reemplazada por interacciones con una base de datos u otro medio de almacenamiento.
4. **Main**: Simula el uso del DAO para realizar operaciones CRUD. Primero se agregan dos usuarios, luego se actualiza uno de ellos, se consulta por ID, se elimina uno y se muestran los usuarios restantes.

Implementación del DAO usando JDBC

El **PreparedStatement** en Java es una clase proporcionada por la API JDBC (Java Database Connectivity) que se utiliza para ejecutar consultas SQL precompiladas con parámetros. A diferencia de las consultas SQL simples (Statement), **PreparedStatement** ofrece ventajas importantes, como la seguridad frente a ataques de **inyección SQL** y una mejor **eficiencia** en la ejecución de consultas repetidas, ya que el SQL se compila solo una vez y puede reutilizarse con diferentes parámetros.

Ventajas de `PreparedStatement`:

UTEC
Universidad Tecnológica

1. **Seguridad contra inyección SQL**: Los parámetros se definen de manera separada del código SQL, lo que evita que un usuario malintencionado manipule las consultas.
2. **Mejor rendimiento**: Las consultas se precompilan, lo que es beneficioso cuando se ejecuta la misma consulta varias veces con diferentes valores.
3. **Legibilidad y reutilización**: Puedes reutilizar la misma consulta con diferentes parámetros sin tener que escribir múltiples versiones de la misma consulta.

Ejemplo básico de uso de `PreparedStatement`

Vamos a trabajar con una base de datos de usuarios y realizar operaciones típicas como inserción, actualización y eliminación usando `PreparedStatement`.

1. Inserción de un nuevo usuario en la base de datos

Supongamos que tenemos una tabla `usuarios` con las columnas `id`, `nombre`, `email` y `password`.

```
CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(50),
    email VARCHAR(50),
    password VARCHAR(50)
);
```

Ahora, veamos cómo insertar un nuevo registro usando `PreparedStatement`:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class UsuarioDAO {
    private static final String INSERT_SQL = "INSERT INTO usuarios
(nombre, email, password) VALUES (?, ?, ?)";

    public void agregarUsuario(String nombre, String email, String
password) {
        try (Connection connection =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/mi_base_de
_datos", "postgres", "admin");
```

```
             PreparedStatement preparedStatement =
connection.prepareStatement(INSERT_SQL)) {

            preparedStatement.setString(1, nombre);  // Asignar el
nombre
            preparedStatement.setString(2, email);   // Asignar el email
            preparedStatement.setString(3, password); // Asignar la
contraseña

            int filasAfectadas = preparedStatement.executeUpdate(); //
Ejecuta la inserción
            System.out.println("Se ha insertado " + filasAfectadas + "
fila(s).");

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

En este ejemplo, la consulta SQL contiene **placeholders** (`?`), que luego son reemplazados por los valores que pasamos a través de `setString()`. Puedes usar otros métodos como `setInt()`, `setDate()`, etc., para diferentes tipos de datos.

2. Actualización de un registro

Veamos ahora cómo actualizar el email de un usuario existente:

```
public class UsuarioDAO {
    private static final String UPDATE_SQL = "UPDATE usuarios SET email
= ? WHERE id = ?";

    public void actualizarEmail(int id, String nuevoEmail) {
        try (Connection connection =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/mi_base_de
_datos", "postgres", "admin");
             PreparedStatement preparedStatement =
connection.prepareStatement(UPDATE_SQL)) {

            preparedStatement.setString(1, nuevoEmail);  // Asignar el
nuevo email
```

```
            preparedStatement.setInt(2, id);             // Asignar el
ID del usuario

            int filasAfectadas = preparedStatement.executeUpdate(); //
Ejecutar la actualización
            System.out.println("Se ha actualizado " + filasAfectadas + "
fila(s).");

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Aquí estamos actualizando el correo electrónico de un usuario en función de su `id`. Nuevamente, `PreparedStatement` ayuda a que el código sea más seguro y eficiente.

3. Eliminación de un registro

Por último, veamos cómo eliminar un usuario de la base de datos utilizando su `id`.

```
public class UsuarioDAO {
    private static final String DELETE_SQL = "DELETE FROM usuarios WHERE
id = ?";

    public void eliminarUsuario(int id) {
        try (Connection connection =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/mi_base_de
_datos", "postgres", "admin");
             PreparedStatement preparedStatement =
connection.prepareStatement(DELETE_SQL)) {

            preparedStatement.setInt(1, id);  // Asignar el ID del
usuario que se eliminará

            int filasAfectadas = preparedStatement.executeUpdate(); //
Ejecutar la eliminación
            System.out.println("Se ha eliminado " + filasAfectadas + "
fila(s).");

        } catch (SQLException e) {
```

```
            e.printStackTrace();
        }
    }
}
```

4. Consulta de un registro

También podemos usar `PreparedStatement` para realizar consultas a la base de datos. Supongamos que queremos obtener un usuario específico por su `id`:

```
import java.sql.*;

public class UsuarioDAO {
    private static final String SELECT_SQL = "SELECT * FROM usuarios
WHERE id = ?";

    public Usuario obtenerUsuarioPorId(int id) {
        Usuario usuario = null;

        try (Connection connection =
DriverManager.getConnection("jdbc:postgresql://localhost:5432/mi_base_de
_datos", "postgres", "admin");
             PreparedStatement preparedStatement =
connection.prepareStatement(SELECT_SQL)) {

            preparedStatement.setInt(1, id);  // Asignar el ID

            ResultSet resultSet = preparedStatement.executeQuery();  //
Ejecutar la consulta

            if (resultSet.next()) {
                // Crear un objeto Usuario con los datos obtenidos
                usuario = new Usuario(
                    resultSet.getInt("id"),
                    resultSet.getString("nombre"),
                    resultSet.getString("email"),
                    resultSet.getString("password")
                );
            }
```

UTEC
Universidad Tecnológica

```
        } catch (SQLException e) {
            e.printStackTrace();
        }

        return usuario;
    }
}
```

En este ejemplo, `executeQuery()` devuelve un `ResultSet`, que contiene los resultados de la consulta. Luego, recorremos el `ResultSet` para extraer los datos.

A continuación, creamos una **implementación concreta del DAO** utilizando JDBC para interactuar con una base de datos relacional.

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class UsuarioDAOImpl implements UsuarioDAO {

    private Connection connection;

    public UsuarioDAOImpl(Connection connection) {
        this.connection = connection;
    }

    @Override
    public void agregarUsuario(Usuario usuario) {
        String query = "INSERT INTO usuarios (nombre, email, password)
VALUES (?, ?, ?)";
        try (PreparedStatement statement =
connection.prepareStatement(query)) {
            statement.setString(1, usuario.getNombre());
            statement.setString(2, usuario.getEmail());
            statement.setString(3, usuario.getPassword());
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
```

UTEC
Universidad Tecnológica

```
    }

    @Override
    public Usuario obtenerUsuarioPorId(int id) {
        String query = "SELECT * FROM usuarios WHERE id = ?";
        Usuario usuario = null;
        try (PreparedStatement statement =
connection.prepareStatement(query)) {
            statement.setInt(1, id);
            ResultSet resultSet = statement.executeQuery();
            if (resultSet.next()) {
                usuario = new Usuario(
                    resultSet.getInt("id"),
                    resultSet.getString("nombre"),
                    resultSet.getString("email"),
                    resultSet.getString("password")
                );
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return usuario;
    }

    @Override
    public List<Usuario> obtenerTodosLosUsuarios() {
        String query = "SELECT * FROM usuarios";
        List<Usuario> usuarios = new ArrayList<>();
        try (Statement statement = connection.createStatement();
             ResultSet resultSet = statement.executeQuery(query)) {
            while (resultSet.next()) {
                Usuario usuario = new Usuario(
                    resultSet.getInt("id"),
                    resultSet.getString("nombre"),
                    resultSet.getString("email"),
                    resultSet.getString("password")
                );
                usuarios.add(usuario);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
```

UTEC
Universidad Tecnológica

```
        return usuarios;
    }

    @Override
    public void actualizarUsuario(Usuario usuario) {
        String query = "UPDATE usuarios SET nombre = ?, email = ?,
password = ? WHERE id = ?";
        try (PreparedStatement statement =
connection.prepareStatement(query)) {
            statement.setString(1, usuario.getNombre());
            statement.setString(2, usuario.getEmail());
            statement.setString(3, usuario.getPassword());
            statement.setInt(4, usuario.getId());
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    @Override
    public void eliminarUsuario(int id) {
        String query = "DELETE FROM usuarios WHERE id = ?";
        try (PreparedStatement statement =
connection.prepareStatement(query)) {
            statement.setInt(1, id);
            statement.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Uso del Patrón DAO en un Servicio

Podemos usar el patrón DAO en una capa de servicio para manejar la lógica de negocio. Este servicio interactuará con el DAO para realizar operaciones CRUD sobre los usuarios.

```
public class UsuarioService {
    private UsuarioDAO usuarioDAO;
```

```
    public UsuarioService(UsuarioDAO usuarioDAO) {
        this.usuarioDAO = usuarioDAO;
    }

    public void agregarUsuario(Usuario usuario) {
        usuarioDAO.agregarUsuario(usuario);
    }

    public Usuario obtenerUsuarioPorId(int id) {
        return usuarioDAO.obtenerUsuarioPorId(id);
    }

    public List<Usuario> obtenerTodosLosUsuarios() {
        return usuarioDAO.obtenerTodosLosUsuarios();
    }

    public void actualizarUsuario(Usuario usuario) {
        usuarioDAO.actualizarUsuario(usuario);
    }

    public void eliminarUsuario(int id) {
        usuarioDAO.eliminarUsuario(id);
    }
}
```

Ejemplo de Uso

A continuación te muestro cómo se usaría este DAO en una aplicación.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Main {
    public static void main(String[] args) {
        // Conexión a la base de datos
        String url =
"jdbc:postgresql://localhost:5432/mi_base_de_datos";
        String username = "postgres";
        String password = "admin";
```

UTEC
Universidad Tecnológica

```
        try (Connection connection = DriverManager.getConnection(url,
username, password)) {
            // Crear instancia del DAO
            UsuarioDAO usuarioDAO = new UsuarioDAOImpl(connection);
            UsuarioService usuarioService = new
UsuarioService(usuarioDAO);

            // Crear y agregar un nuevo usuario
            Usuario nuevoUsuario = new Usuario(0, "Juan Pérez",
"juan.perez@example.com", "1234");
            usuarioService.agregarUsuario(nuevoUsuario);

            // Obtener y mostrar todos los usuarios
            for (Usuario usuario :
usuarioService.obtenerTodosLosUsuarios()) {
                System.out.println(usuario.getNombre() + " - " +
usuario.getEmail());
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Resumen

El patrón DAO proporciona una manera organizada de separar la lógica de acceso a datos del resto de la aplicación, facilitando el mantenimiento y la escalabilidad del código. En este ejemplo, usamos el patrón para manejar usuarios en una base de datos, pero puedes aplicarlo a cualquier entidad en tu sistema.